

Complete Django Backend Architecture

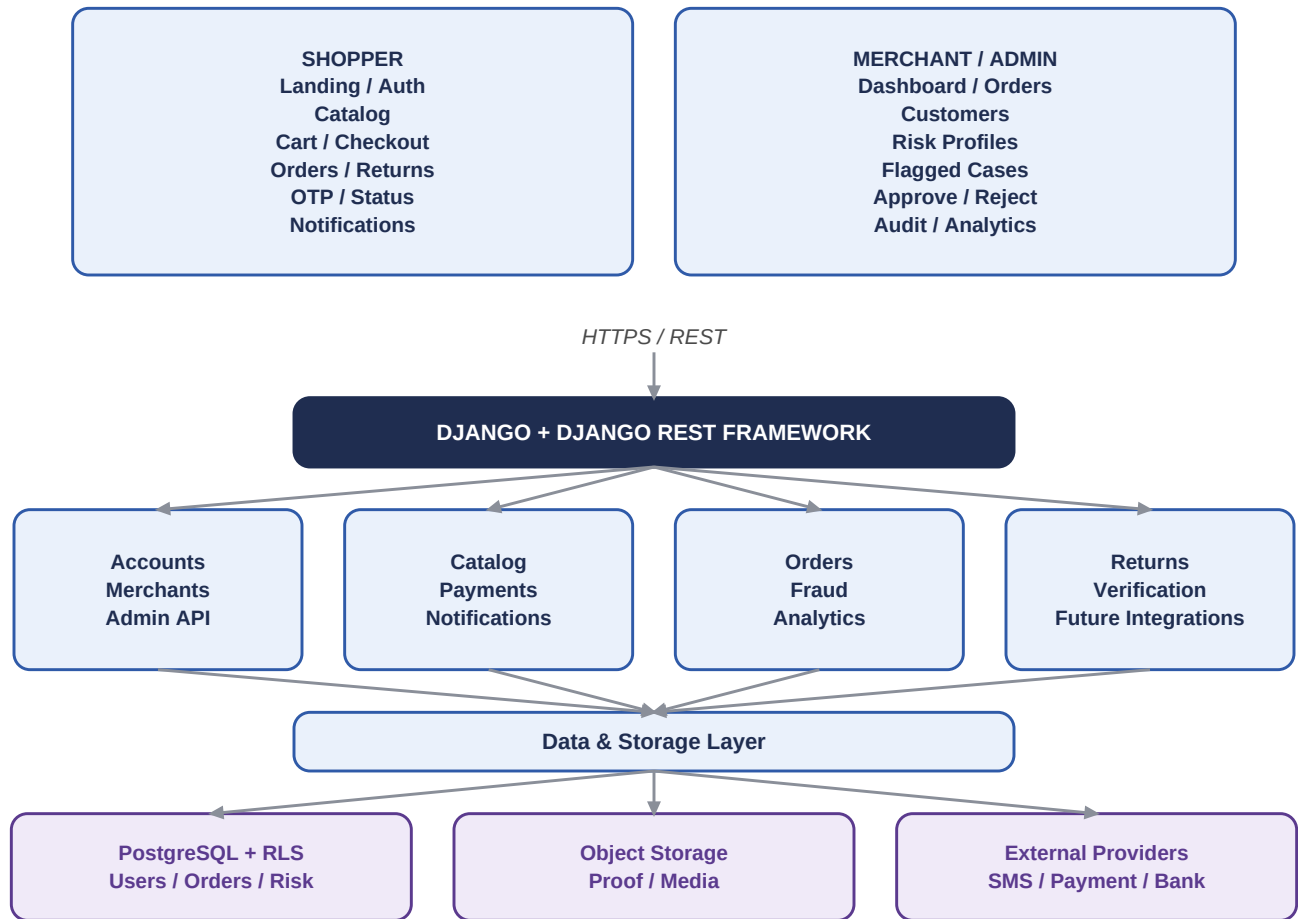
React.js Frontend → Django + DRF → PostgreSQL

Current MVP + Payment/Invoice Layer + Future Features

Visual implementation blueprint with architecture frames, data-flow diagrams, frontend-to-backend mapping, fraud workflows, payment and invoice notification architecture, database design and future expansion paths.

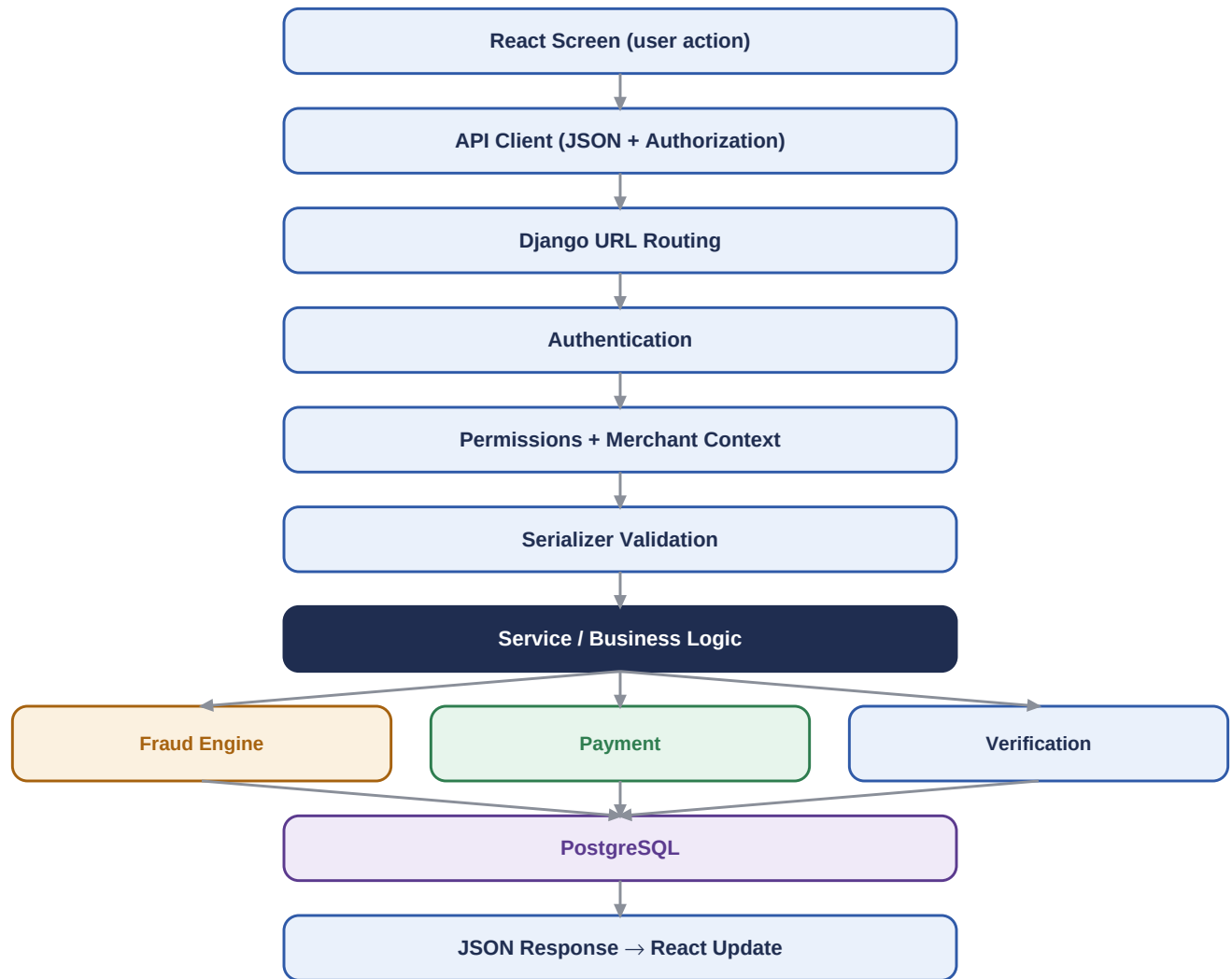
Revision note: this version adds a dedicated Payment Gateway, Webhook, Failure Handling and Automated Invoice Email Architecture section (Section 5). All previously documented sections are preserved below; section numbers from 5 onward have shifted by one to make room for it.

1. Complete Platform Architecture



React shopper and merchant surfaces call the same Django + DRF API layer, which is backed by PostgreSQL, object storage and external providers.

2. Frontend → Backend Data Flow

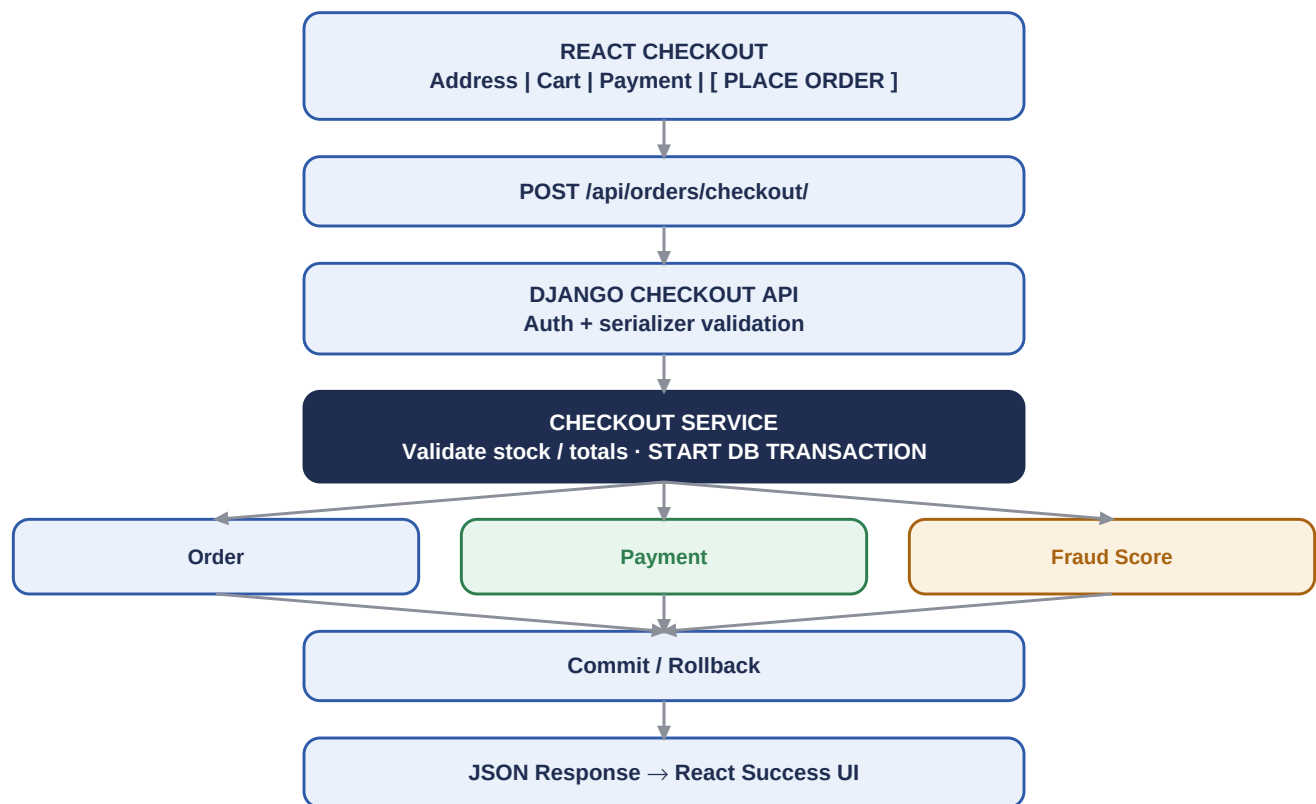


3. Frontend Screen to Backend Map

Frontend frame	Backend API / module
Login / Register	accounts → authentication, role creation, sessions/tokens
Catalog / Product Details	catalog → products, categories, inventory
Cart / Checkout	orders + payments + fraud → atomic order creation
Payment Result Screen	payments → webhook-verified status, invoice link
OTP Screen	verification → send/verify, attempt history, re-score
Order History	orders → paginated tenant/user-scoped history
Return Request	returns + fraud → validation, risk scoring, timeline
Return Status	returns → events, proof, decision state

Frontend frame	Backend API / module
Merchant Dashboard	admin_api → summary metrics and queues
Customers / Risk Profile	fraud + admin_api → risk history and evidence
Flagged Case	fraud + returns → context, review, decision
Audit Log	audit service → immutable admin actions
Future Analytics	analytics → aggregates, trends, anomaly outputs

4. Checkout Frame and Backend Flow



Payment record creation and the payment/invoice notification flow that follows checkout are detailed in Section 5.

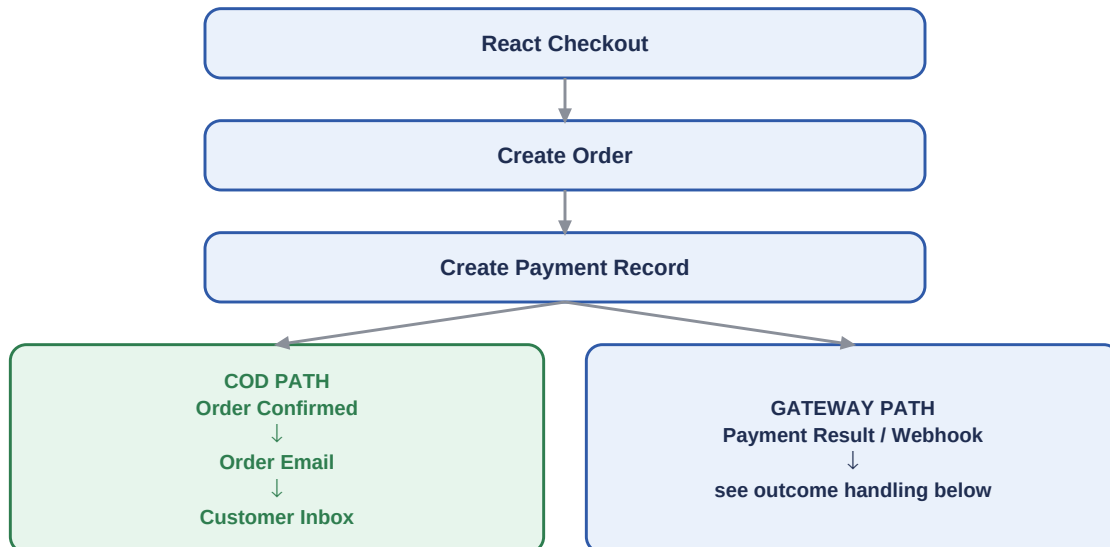
5. Payment Gateway, Webhooks, Failure Handling & Automated Invoice Email Architecture

This section defines how payment results are captured, how customers are notified for every outcome, how invoices are generated and delivered, and when the real payment gateway should be introduced relative to the rest of the backend build.

5.1 Payment & Invoice Notification Matrix

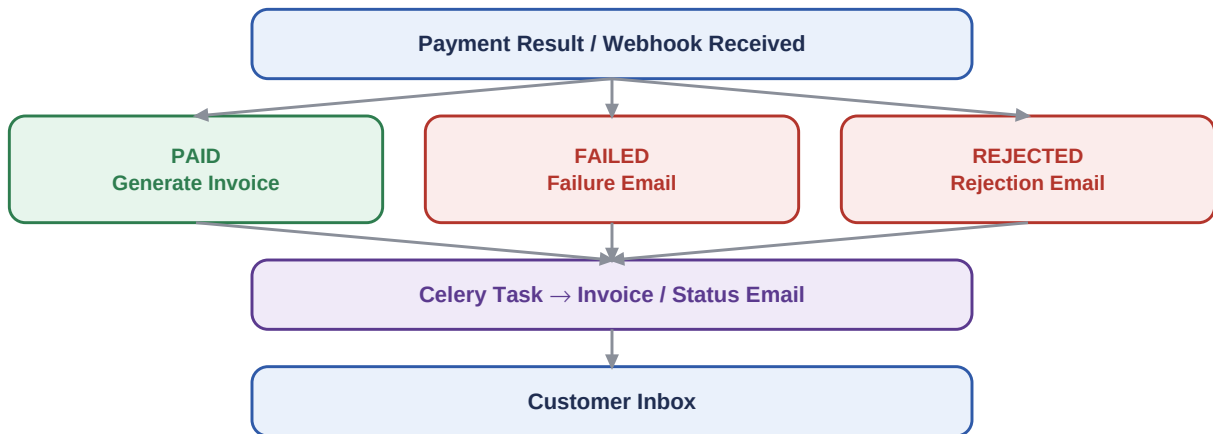
Event	Order status	Payment status	Customer email
Payment initiated	Pending	Processing	Optional
Payment successful	Confirmed / Processing	Paid	Payment success + invoice
Payment failed	Payment pending / Failed	Failed	Payment failure
Payment rejected / declined	Pending	Rejected	Payment rejected
COD order placed	Confirmed	COD pending	Order confirmation

5.2 Recommended Checkout → Payment → Notification Flow



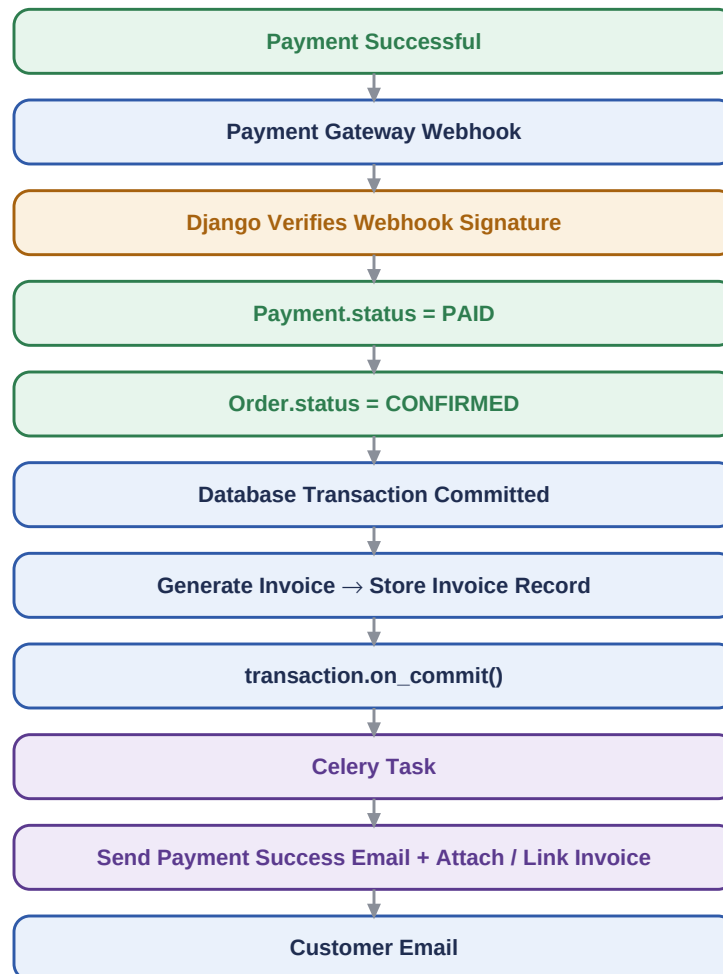
COD orders confirm immediately and email the customer without waiting on a gateway. Gateway orders continue into the outcome handling below.

Gateway Payment Outcome Handling



Gateway orders wait for a verified webhook result before the order or invoice state changes.

5.3 Invoice Email Flow (Online Prepaid Orders)



For COD orders, send an order confirmation immediately; generate the final payment invoice once payment is actually collected, per business/payment rules.

5.4 Never Trust the Frontend Payment Message Alone

Even if React shows a “Payment Successful” message, Django must not mark a payment as paid from that response alone. The gateway’s signed webhook is the single source of truth — this prevents forged payment-success requests from being sent directly to the API.

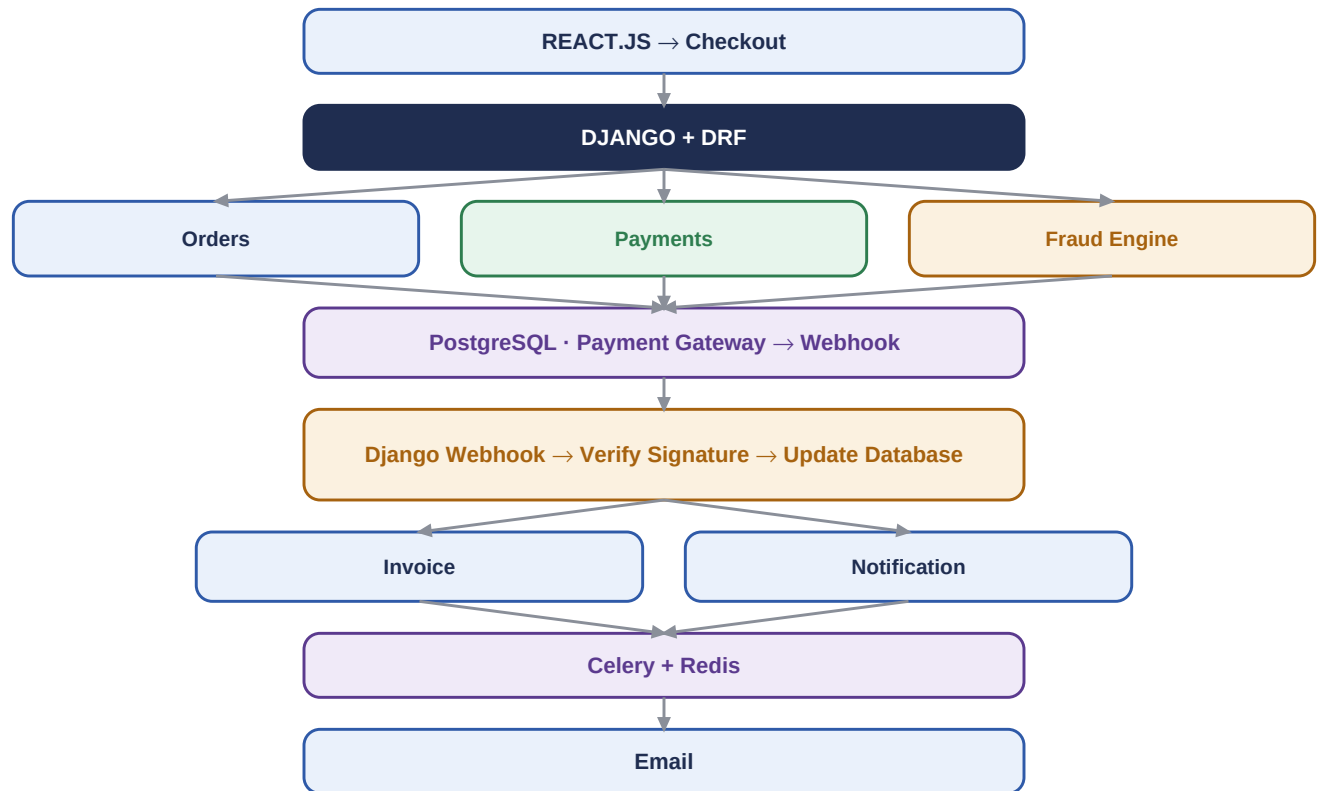


5.5 Payment / Invoice Data Model Additions

```
Order
|-- Payment (order_id, gateway, gateway_payment_id, amount, status, failure_reason)
| |-- PaymentEvent (payment_id, event_type, gateway_event_id, payload, created_at)
|
|-- Invoice (order_id, invoice_number, invoice_url, status, generated_at)
|
|-- EmailDelivery (user_id, event_type, related_order_id, status, sent_at)
```

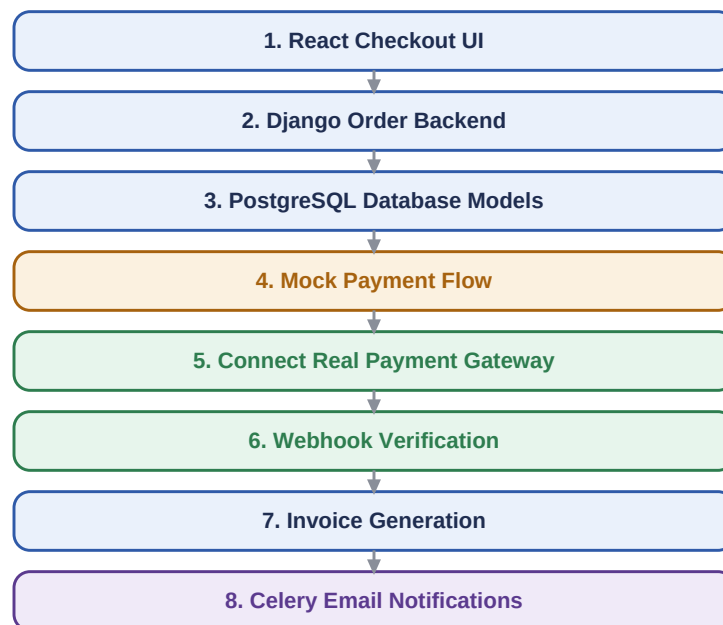
Payment, PaymentEvent, Invoice and EmailDelivery extend the Fraud Data Model in Section 7 — they hang off the same Order entity used by returns and fraud scoring.

5.6 End-to-End Architecture With Payment Layer

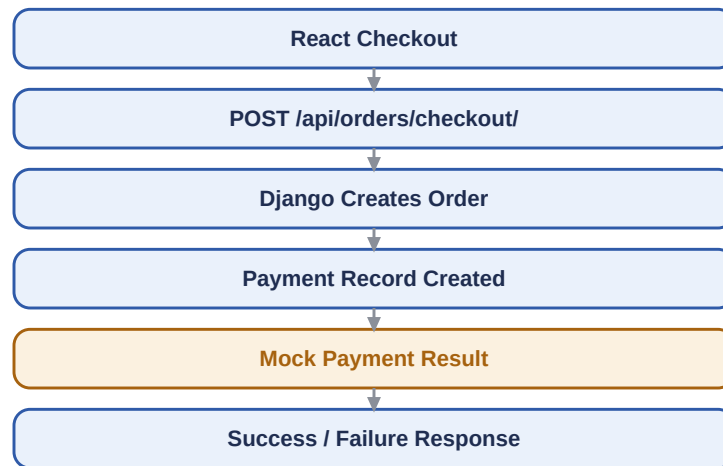


5.7 When to Introduce the Real Payment Gateway

Recommendation: do not implement the real payment gateway first. Build the surrounding backend and prove the flow with a mock payment service before connecting a live provider.

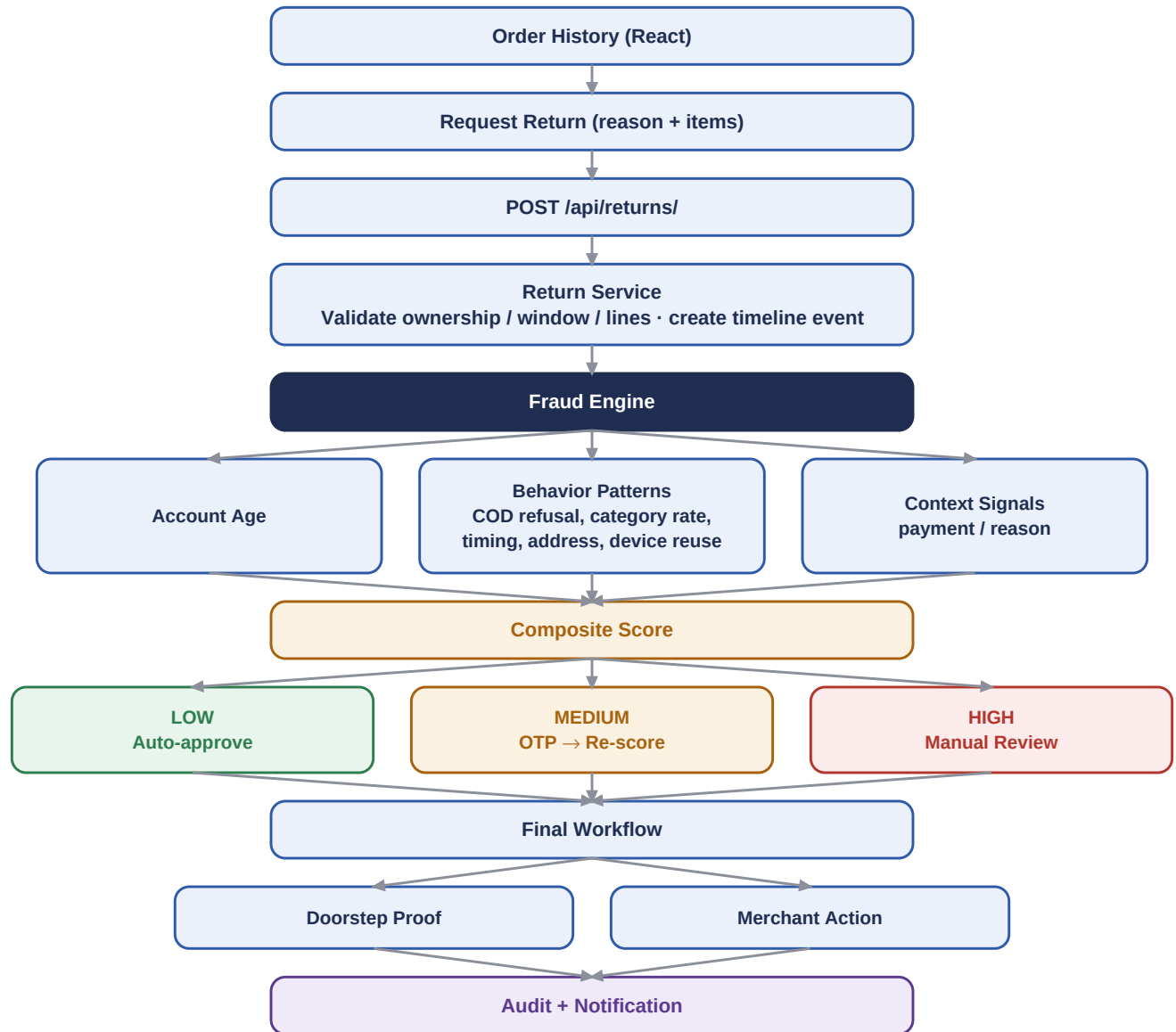


5.8 Mock-First Development Sequence



This lets the whole application be built and tested before a real payment provider is connected. New modules: orders/, payments/, invoices/, notifications/.

6. Complete Return + Fraud Flow



7. Fraud Data Model

```
Merchant
|-- Product -- Category
|-- Customer / User
| |-- CustomerRiskProfile
| | |-- RiskScoreEvent
| | |-- VerificationEvent
|-- Order
|-- OrderItem
|-- Payment (see Section 5.5)
| |-- PaymentEvent
|-- Invoice
|-- EmailDelivery
|-- ReturnRequest
```

```
|-- ReturnLine
|-- ReturnEvent
|-- DoorstepProof
|-- ReviewDecision

AuditLog -> references sensitive merchant/admin actions
```

8. Django Application Architecture

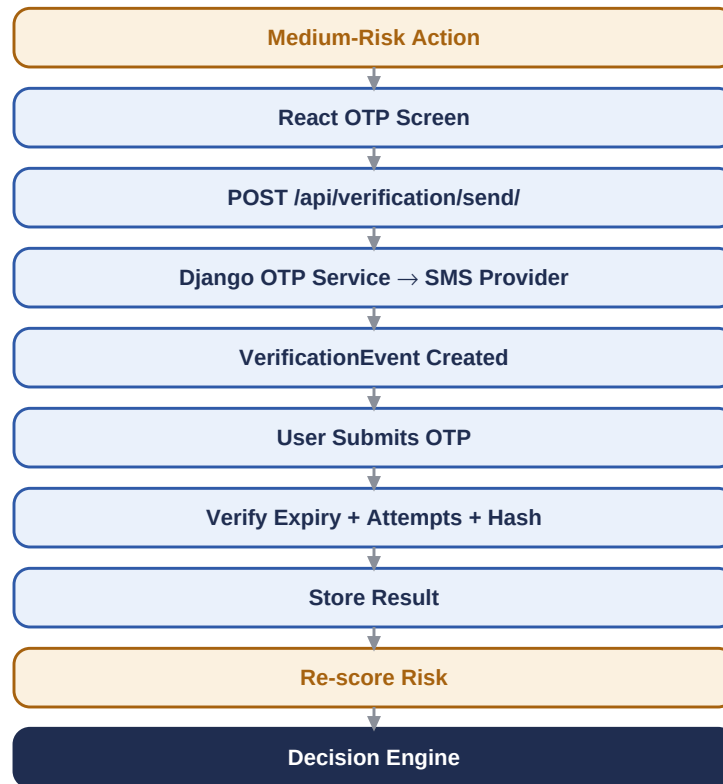
```
backend/
|-- config/
| |-- settings/
| |-- urls.py
| |-- middleware.py
|-- common/
| |-- permissions.py
| |-- exceptions.py
| |-- tenancy.py
|-- accounts/
|-- merchants/
|-- catalog/
|-- cart/
|-- orders/
|-- payments/ # gateway integration, webhooks
|-- invoices/ # NEW: invoice generation + storage
|-- returns/
|-- fraud/
| |-- services/
| | |-- risk_engine.py
| | |-- signal_extractors.py
| | |-- decision_engine.py
| |-- tasks/
|-- verification/
|-- notifications/ # payment, invoice, order, return emails
|-- proof/
|-- audit/
|-- analytics/
|-- delivery_risk/ # future
|-- bank_validation/ # future
|-- threshold_learning/ # future
|-- admin_api/
```

9. API Architecture

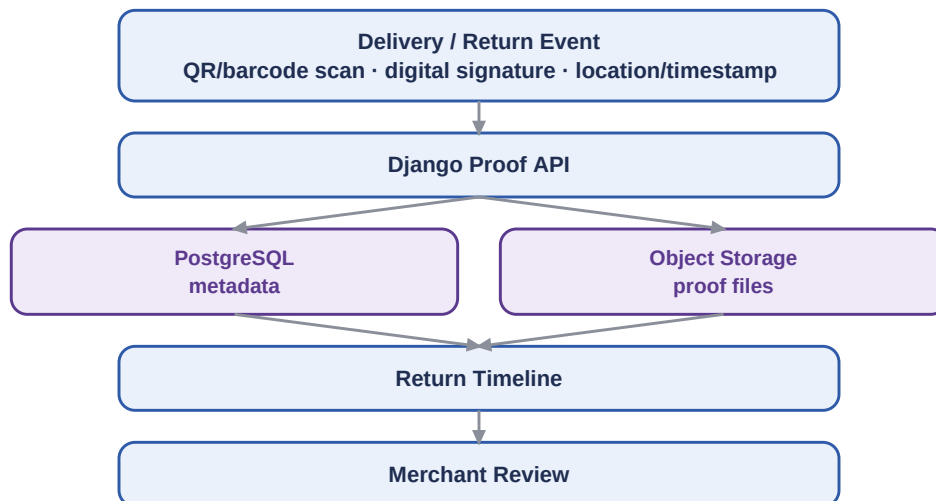
Domain	Representative endpoints
Authentication	POST /api/auth/register/ login/ refresh/ logout/
Products	GET /api/products/ GET /api/products/{id}/
Cart	GET/POST/PATCH /api/cart/
Checkout	POST /api/orders/checkout/

Domain	Representative endpoints
Orders	GET /api/orders/ GET /api/orders/{id}/
Payments	POST /api/payments/webhook/ GET /api/payments/{id}/
Invoices	GET /api/invoices/{id}/ GET /api/orders/{id}/invoice/
Returns	POST /api/returns/ GET /api/returns/{id}/ GET /timeline/
OTP	POST /api/verification/send/ POST /verify/
Proof	POST /api/returns/{id}/proof/
Merchant	GET /api/admin/dashboard/ /orders/ /customers/
Risk Review	GET /api/admin/flagged-cases/ POST /approve/ /reject/
Future	/api/bank-validation/ /delivery-risk/ /analytics/

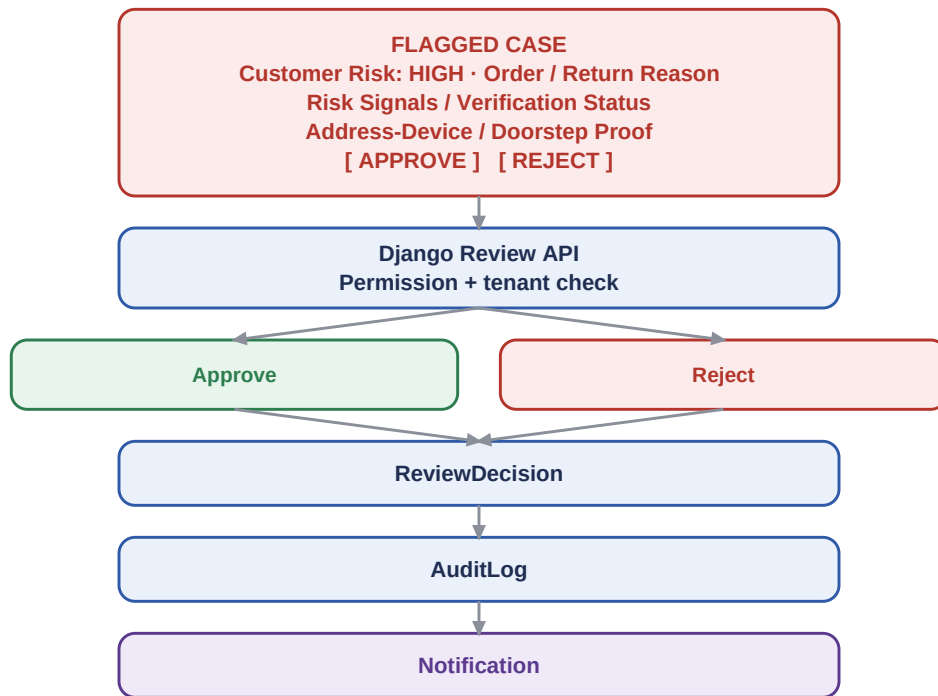
10. Verification and OTP Flow



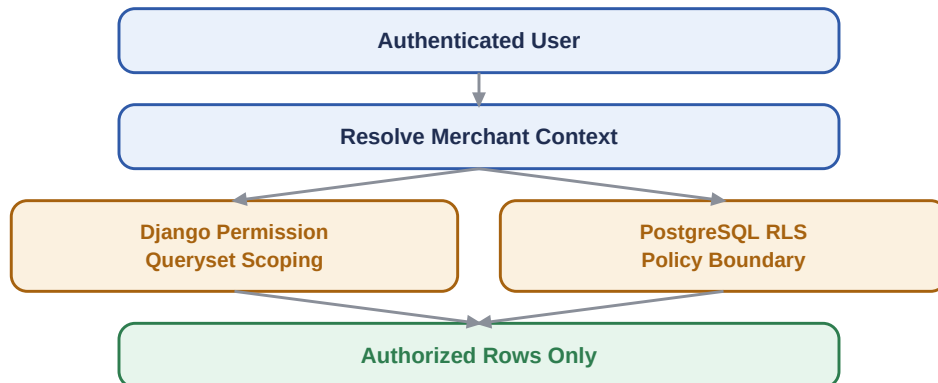
11. Doorstep Proof Data Flow



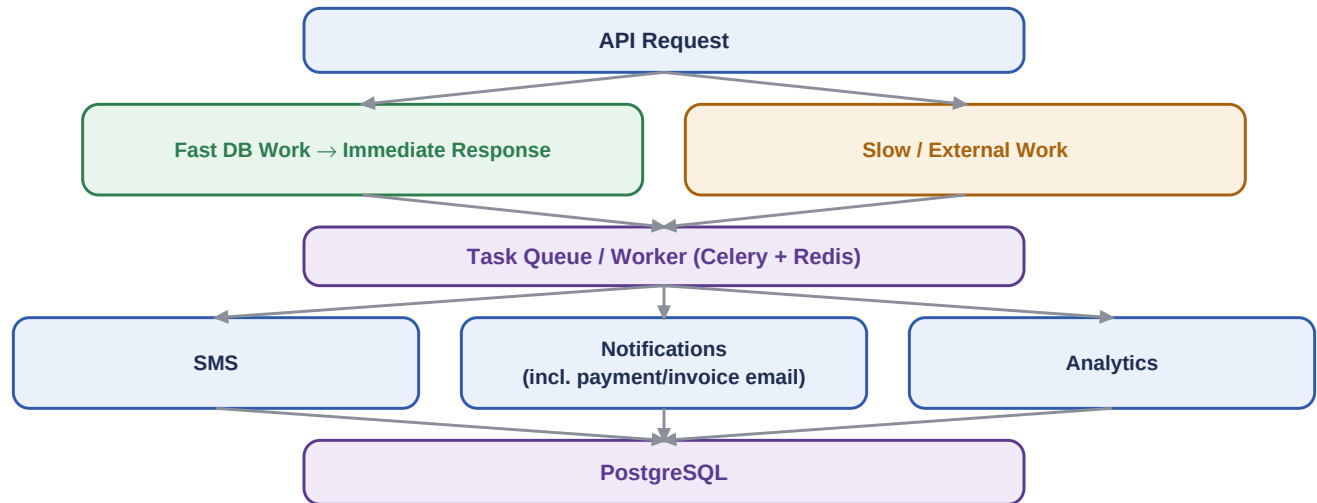
12. Merchant Review Frame



13. Multi-Tenant Security Design

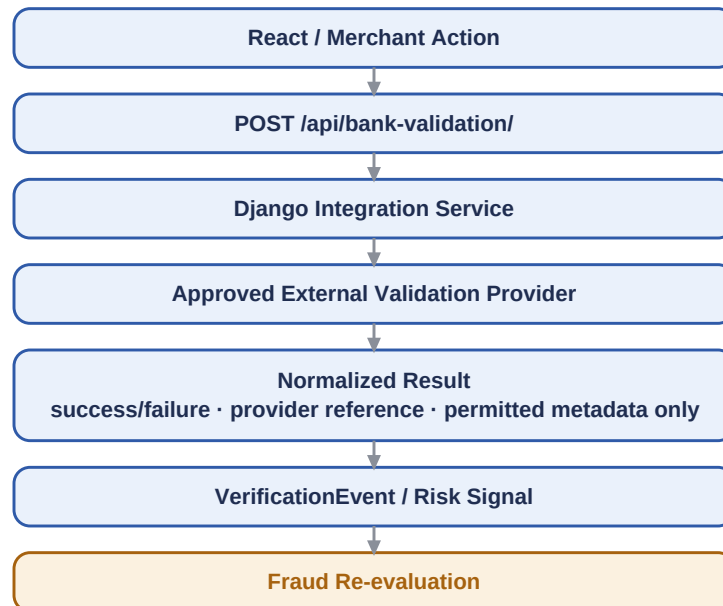


14. Background Processing Architecture



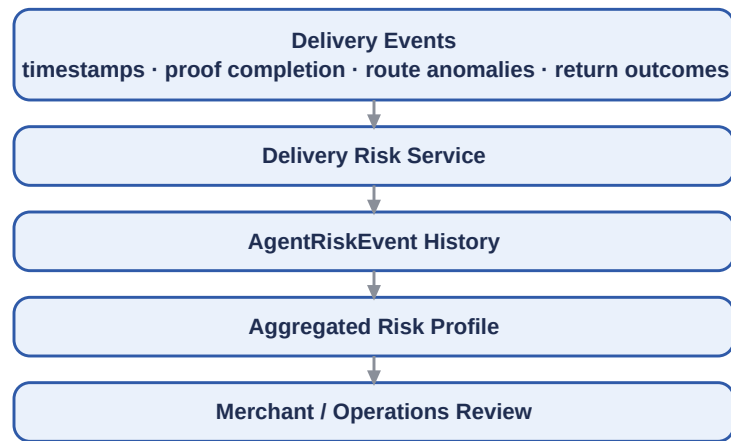
Use this pattern for SMS, notifications, payment/invoice emails, scheduled fraud analytics, future anomaly calculations and other slow integrations.

15. Future Feature: Bank / UPI Validation



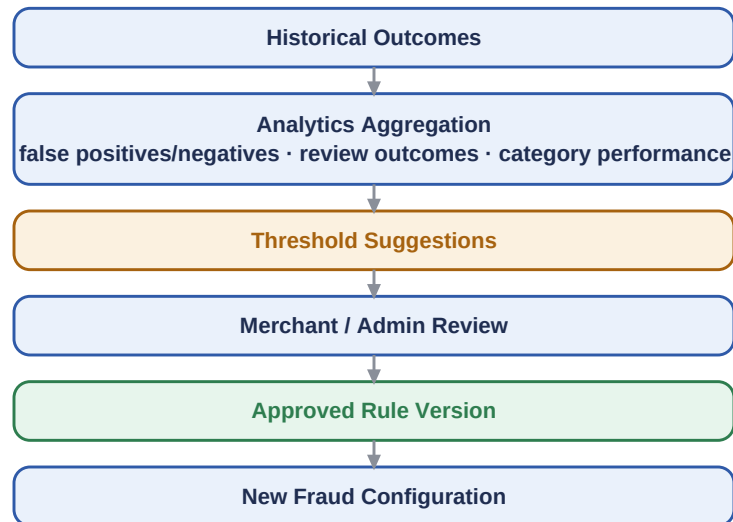
Implementation details depend on the provider and legal/privacy requirements; the backend should isolate provider-specific code behind an adapter/service.

16. Future Feature: Delivery-Agent Risk Analysis



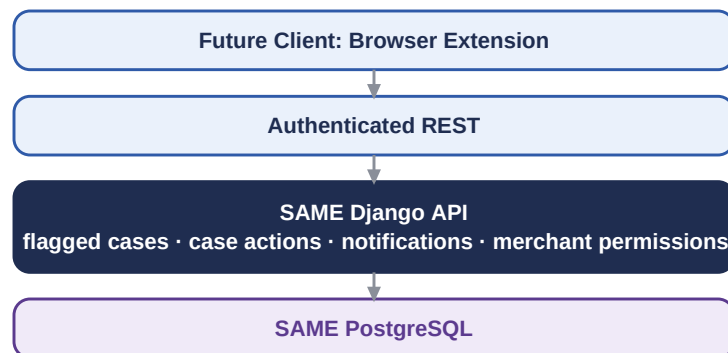
Keep delivery-agent analysis separate from shopper risk so evidence, permissions and review logic remain understandable.

17. Future Feature: Self-Tuning Thresholds



Do not automatically change production thresholds without review. Persist configuration versions so every decision can be traced to the rule set that produced it.

18. Future Extension Architecture



No duplicate fraud engine inside the extension. The extension remains a future client surface; Django remains the single backend and fraud decision source.

19. Performance Architecture

Area	Design rule
React lists	Server-side pagination and filtering
ORM	select_related / prefetch_related and measured indexes
API payload	Separate compact list serializers from detailed serializers
Caching	Cache stable reads; invalidate after writes
External APIs	Background workers and retries

Area	Design rule
Database writes	Atomic transactions
Fraud scoring	Efficient signal queries; avoid repeated history scans
Payments/Invoices	Idempotent webhook handling; retry queue for email delivery
Analytics	Pre-aggregation and scheduled jobs for heavy reporting
Files	Object storage; never embed large media in JSON
Monitoring	Request IDs, latency, error logs and health checks

20. Error and Response Frame

Success

```
{
  "data": {...},
  "meta": {
    "request_id": "...",
    "pagination": {...}
  }
}
```

Error

```
{
  "error": {
    "code": "RETURN_NOT_ELIGIBLE",
    "message": "This order cannot be returned.",
    "field_errors": {},
    "request_id": "..."
  }
}
```

Payment webhook error (example)

```
{
  "error": {
    "code": "PAYMENT_SIGNATURE_INVALID",
    "message": "Webhook signature verification failed.",
    "field_errors": {},
    "request_id": "..."
  }
}
```

21. Complete Development Roadmap

Phase	Scope
Phase 1 — Foundation	Django + PostgreSQL + environment + custom user + merchants
Phase 2 — Core Commerce	Catalog → Cart → Checkout → Orders → Mock Payments
Phase 3 — Payment & Invoice	Gateway integration → webhooks → invoice generation → Celery emails
Phase 4 — Returns	ReturnRequest → ReturnLine → Timeline → Proof
Phase 5 — Fraud	Signals → Composite scoring → Risk events → Decisions
Phase 6 — Verification	OTP → verification events → re-scoring

Phase	Scope
Phase 7 — Merchant Operations	Dashboard → Customers → Flagged cases → Actions → Audit
Phase 8 — Performance & Security	RLS → indexes → caching → background workers → testing
Phase 9 — Future Integrations	Bank/UPI → delivery risk → analytics → threshold suggestions
Phase 10 — Future Client	Chrome extension using the same secured Django APIs

Payment gateway integration (Phase 3) intentionally comes right after core commerce is provable end-to-end with a mock payment service — see Section 5.7.

22. Final Implementation Contract

- 1 Identify the React screen.
- 2 Identify the user action.
- 3 Define endpoint and HTTP method.
- 4 Define request serializer.
- 5 Validate permissions and tenant context.
- 6 Call a focused service layer.
- 7 Use an atomic transaction where state changes are coupled.
- 8 Run fraud / payment / verification logic when applicable.
- 9 Verify payment webhooks by signature before trusting any status change.
- 10 Persist normalized history, invoice and audit records.
- 11 Return a stable JSON response.
- 12 Update React using the response.
- 13 Add unit, API, tenant-isolation and regression tests.

Final Result

This blueprint provides a complete backend direction: the current commerce and return-fraud platform, a dedicated payment gateway / webhook / invoice / notification layer, and future bank/UPI validation, delivery-agent risk analysis, analytics, threshold suggestions and a future browser-extension client. The visual frames and flow diagrams are designed to make the implementation sequence easier to follow when using the document as a build specification.